

1 Asymptomatic Analysis

- 1.1 Master Theorem**
- $T(n) = aT(\frac{n}{b}) + f(n)$
 - nLeaves = n^d where $d = \log_b a$ is the critical exponent
 - Case 1: $f(n) \in O(n^{d-\epsilon}) : \epsilon > 0$ (NR cost « nLeaves)
 $\rightarrow T(n) \in \Theta(n^d)$
NR Cost « nLeaves $\rightarrow$ Most computation: base cases
 - Case 2: $f(n) \in \Theta(n^d \log^k n : k \geq 0)$ (NR cost $\approx$ nLeaves)
 $\rightarrow T(n) \in \Theta(n^d \log^{k+1} n = f(n) \times \log(n)) =$ NR Cost x Height
NR Cost $\approx$ nLeaves $\rightarrow$ Computation shared evenly
 - Case 3: $f(n) \in \Omega(n^{d+\epsilon} : \epsilon > 0) \wedge (af(\frac{n}{b}) \leq cf(n)) : c < 1$ (NR cost » nLeaves)
 $\rightarrow T(n) \in \Theta(f(n))$
NR Cost » nLeaves $\rightarrow$ Most computation: higher levels
 - Note that the first condition in Case 3 is redundant. Main(Regularity) condition is that the combined subproblem cost $\leq$ current problem non-recurring cost.

- 1.2 Helpers**
- 1.2.1 Stirling's Approx**
- $\log(n!) \approx \Theta(\log n) + \Theta(n \log n) - \Theta(n)$
- 1.2.2 Log rule**
- $\log \log_a b = c \rightarrow b = a^c$
- 1.2.3 AP/GP**
- Where: a_1 = first term, a_n = n -th term, d = common difference, r = common ratio, S_n = sum of first n terms
- | | |
|--|---|
| <ul style="list-style-type: none">Arithmetic$a_n = a_1 + (n - 1)d$$S_n = \frac{n}{2} [2a_1 + (n - 1)d]$$S_n = \frac{n}{2} (a_1 + a_n)$ | <ul style="list-style-type: none">Geometric$a_n = a_1 r^{n-1}$$S_n = a_1 \frac{1-r^n}{1-r}$$S_\infty = \frac{a_1}{1-r}, \quad r < 1$ |
|--|---|

2 Dynamic Programming Optimal Substructures

- 2.1 Fibonacci**
- Input: Integer n
 - Recurrence formula:
$$F(n) = \begin{cases} n & \text{if } n \leq 1 \\ F(n-1) + F(n-2) & \text{otherwise} \end{cases}$$
 - Recurrence state: 1D array or memoization table of size $n + 1$
 - Subproblem dependency: $F(n)$ depends on $F(n-1)$ and $F(n-2)$
 - $\rightarrow$ Time Complexity: $\Theta(n)$ with memoization or bottom-up tabulation
 - $\rightarrow$ Space Complexity: $\Theta(1)$ optimized (iterative) or $\Theta(n)$ unoptimized
- 2.2 Longest Common Subsequence (LCS)**
- Input: Two arrays $A[1..m]$ and $B[1..n]$
 - Recurrence formula $L(i, j)$:
- $$L(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L(i-1, j-1) + 1 & \text{if } i, j > 0 \text{ and } a_i = b_j \\ \max(L(i-1, j), L(i, j-1)) & \text{if } i, j > 0 \text{ and } a_i \neq b_j \end{cases}$$
- Recurrence state: 2D table $L[0..m, 0..n]$
 - Subproblem dependency: $L(i, j)$ depends on diagonal $L(i-1, j-1)$, top $L(i-1, j)$, and left $L(i, j-1)$ cells
 - $\rightarrow$ Space Complexity: $\Theta(mn)$ space ($\Theta(\min(m, n))$ with space optimization)
 - $\rightarrow$ Time Complexity: $\Theta(mn)$ time
- 2.3 0/1-Knapsack**
- Input: n items described as $(weight, value)$ pairs: $\{(w_1, v_1), (w_2, v_2), \dots, (w_n, v_n)\}$ and capacity W
 - Recurrence formula $K(i, j)$ (the value of an optimal solution for the instance $\{(w_1, v_1), \dots, (w_i, v_i)\}$ and capacity j):
- $$K(i, j) = \begin{cases} -\infty & \text{if } j < 0 \\ 0 & \text{if } i = 0 \text{ and } j \geq 0 \\ \max(v_i + K(i-1, j-w_i), K(i-1, j)) & \text{otherwise} \end{cases}$$
- Recurrence state: 2D table $K[0..n, 0..W]$
 - Subproblem dependency: $K(i, j)$ depends on taking item i (if $j \geq w_i$) or skipping item i
 - $\rightarrow$ Time Complexity: $\Theta(nW)$ pseudopolynomial time
 - $\rightarrow$ Space Complexity: $\Theta(nW)$ space ($\Theta(W)$ with rolling array)

- 2.4 Coin-Change Problem**
- Goal: Find the minimum number of coins of denominations $\{d_1, d_2, \dots, d_k\}$ that add up to n cents
 - Recurrence formula $M(n)$ (fewest number of coins needed to change n cents):
$$M(n) = \begin{cases} \infty & \text{if } n < 0 \\ 0 & \text{if } n = 0 \\ 1 + \min_{i \in [1..k]} M(n - d_i) & \text{otherwise} \end{cases}$$
 - Recurrence state: 1D array $M[0..n]$
 - Subproblem dependency: $M(n)$ depends on subproblems of smaller remaining amounts $n - d_i$
 - $\rightarrow$ Time Complexity: $\Theta(n \cdot k)$ time ($\Theta(n)$ if k is treated as constant)
 - $\rightarrow$ Space Complexity: $\Theta(n)$ space ($\Theta(k)$ per state or $\Theta(n)$ aux space)
- 2.5 Longest Increasing Subsequence (LIS)**
- Input: Given an array $A[1..n]$ with n integers, plus two sentinels $A[0] = -\infty$ and $A[n+1] = \infty$
 - Recurrence formula $L(j)$ (length of the longest strictly increasing subsequence of $A[1..j]$ that ends at index j , for $j \in [0..n+1]$, computing $L(n+1) - 1$):
$$L(j) = \begin{cases} 0 & \text{if } j = 0 \\ 1 + \max_{i \in [0..j-1] \text{ and } A[i] < A[j]} L(i) & \text{if } j > 0 \end{cases}$$
 - Recurrence state: 1D array $L[0..n+1]$ where $L[j]$ is length of LIS ending at index j
 - Subproblem dependency: $L(j)$ depends on all valid preceding smaller elements where $A[i] < A[j]$
 - $\rightarrow$ Time Complexity: $\Theta(n^2)$ time ($\Theta(n \log n)$ with binary search / patience sorting)
 - $\rightarrow$ Space Complexity: $\Theta(n)$ space